

Model interface

A hands-on course in building agents by building their harness

Lesson 1

The model is a function

```
TokensOut = Model(TokensIn)
```

The **model interface** is how the harness calls it.

A request goes out

It sends tokens to the model as a **request**.

A response comes back

The model returns tokens as a **response**.

Just like any other API endpoint.

The thinnest of the five primitives

That's because of **where it sits**.

Four primitives live on the harness's side

- **Control flow** decides when to call
- **Input** gathers what goes in
- **Context** decides how it's presented in the request
- **Output** handles the response

The model interface is the boundary

The model interface is the boundary between the harness and the model.

The parentheses

```
Agent = Harness(Model)
```

In **Agent = Harness(Model)**, it's the parentheses.

It comes first

Nothing reaches the model without it, so it comes first.

The worked example

The call at the center of quark, with nothing around it.

It's the whole of `quark.py`: nine lines.

quark.py

```
from anthropic import Anthropic

# — model interface —————
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(**request): return client.messages.create(model=MODEL, **request)

output = call(max_tokens=16384, messages=[{"role": "user", "content": "What's in this directory?"})
print(output.model_dump_json(indent=2))
```

client: where the request goes

```
client = Anthropic()
```

Anthropic's hosted API, through the official SDK.

The official SDK, which reads your key from **ANTHROPIC_API_KEY**.

`call()`: the model interface, all of it

```
MODEL = "claude-sonnet-5-5"  
def call(**request): return client.messages.create(model=MODEL, **request)
```

It sends a request to `MODEL` and returns the response.

call() is the one place

Every later lesson calls the model through this one function.

So the primitive stays in one place.

messages : the tokens in

```
output = call(max_tokens=16384, messages=[{"role": "user", "content": "What's in this directory?"}])
```

A list of messages, each with a **role** and **content** .

max_tokens: the cap on tokens out

```
output = call(max_tokens=16384, messages=[{"role": "user", "content": "What's in this directory?"}])
```

The model stops there, even mid-sentence.

output: the response

```
print(output.model_dump_json(indent=2))
```

The tokens out, plus details about how they were produced.

The **print** shows all of it.

The section heading

```
# — model interface
```

It will stay at the top of `quark.py` through every lesson.

Each lesson adds a section of its own.

Run it

From the root of the repo:

```
uv run lessons/01-model-interface/quark.py
```

One run

Here's one run: the whole response comes back as JSON.

Yours will be worded differently: the model's output varies from run to run.

The long `signature` is shortened in the lesson, and here.

Field: content

```
"content": {
  {
    "signature": "CAGSLv0M8AGS4S4N0CT5d0p4n0p40N1rW0N0N18u018U0N0N0T0N0N0p40N1800...",
    "thinking": "",
    "type": "thinking"
  },
  {
    "citations": null,
    "text": "I can't see your directory. I don't have access to your file system in this conversation, and no files or attachments have been shared.\n\nYou can list the contents yourself with one of these:\n\n- \"ls\" (or \"ls -la\" to include hidden files and details)\n\n- \"Windows Command Prompt\": \"dir\" \n\n- \"Windows PowerShell\": \"Get-Childitem\" (or \"ls\")\n\nIf you paste the output here, I can help you work out what the files are, how the project is organized, or what to do next. If you're using a tool or IDE that is supposed to give me file access, it doesn't seem to be connected here.",
    "type": "text"
  }
}
```

content: the tokens out

A list of blocks.

This model thinks before it answers, so the first block is its **thinking** and the second is the **text** of its reply.

content: the thinking block

By default the thinking text comes back empty, with only a **signature**.

A **signature**: an encrypted copy of the reasoning that only the API can read.

Field: `stop_reason`

```
"stop_reason": "end_turn",
```

Why it stopped.

- `end_turn` means it finished
- `max_tokens` would mean it hit the cap

Field: **usage**

```
"input_tokens": 15,  
"output_tokens": 242,  
"output_tokens_details": {  
  "thinking_tokens": 41  
},
```

usage counts both sides.

usage : both sides

- `input_tokens` is `TokenIn`
- `output_tokens` is `TokenOut`, thinking included

Read the `text`

The model knows the right next step is `ls` .

It just can't take it.

Going further

What else a model interface can be.

The mechanism is always a request out and a response back, but it holds more in other harnesses than it does here.

What else it can be: the request

- **Where the request goes.** A hosted API, a model on your own machine, a gateway in front of several providers.
- **How it travels.** An official SDK, raw HTTP, or a layer that speaks several providers' formats so the rest of the harness doesn't have to.
- **Which model answers.** One model, or a backup that takes over when the first is unavailable.

What else it can be: the response and the rest

- **How the response arrives.** All at once, or streamed piece by piece as it's produced, so a long response can't time out.
- **How many go at once.** One request, or a batch of them.
- **What happens when a request fails.** Retry it, wait out a rate limit, give up after a timeout.
- **The request's settings.** How many tokens out, how much the model thinks first, whether you see a summary of its thinking.

A product on its own

Gateways like **LiteLLM** and **OpenRouter** are this primitive and nothing else.

A harness sends them one request format, and they handle where it goes, which provider and model answer, backups when one is down, retries and rate limits.

Using a gateway

A harness that uses a gateway has handed off its model interface.

model_interface.py

A model interface that does more of that.

```
import anthropic

client = anthropic.Anthropic(timeout=120, max_retries=3)
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]

def call(messages, max_tokens=16384, effort="high", thinking="summarized"):
    for model in MODELS:
        try:
            with client.messages.stream(model=model, max_tokens=max_tokens, output_config={"effort": effort}, thinking={"type": "adaptive", "display": thinking}, messages=messages) as stream:
                return stream.get_final_message()
        except (anthropic.APIConnectionError, anthropic.RateLimitError, anthropic.InternalServerError):
            continue
    raise RuntimeError("no model answered")

reply = call([{"role": "user", "content": "What's in this directory?"}])
print(reply.model_dump_json(indent=2))
```

Failure: `timeout` and `max_retries`

```
client = anthropic.Anthropic(timeout=120, max_retries=3)
```

- `timeout=120` gives up on a request that takes longer than two minutes.
- `max_retries=3` has the SDK retry dropped connections, rate limits and server errors, waiting longer each time.

Which model answers: the next one

```
MODELS = ["claude-sonnet-5-5", "claude-opus-5-5"]
```

```
    except (anthropic.APIConnectionError, anthropic.RateLimitError, anthropic.InternalServerError):  
        continue  
    raise RuntimeError("no model answered")
```

If every retry fails, `call` moves on to the next model in `MODELS`.

How the response arrives: **stream**

```
with client.messages.stream(model=model, max_tokens=max_tokens, output_config={"effort": effort}, thinking={"type": "adaptive", "display": thinking}, messages=messages) as stream:  
    return stream.get_final_message()
```

stream receives the response as it's produced, so a long one can't time out.

get_final_message() hands back the whole response once it's done.

Streaming prints nothing

Nothing is printed along the way.

Showing it to a person would be **output**.

The request's settings

```
def call(messages, max_tokens=16384, effort="high", thinking="summarized"):
```

`max_tokens`, `effort` (how much the model thinks first) and `thinking` are arguments the caller can set.

thinking="summarized"

"summarized" asks for a readable summary of the model's thinking instead of the empty default.

The raw thinking itself is never returned.

Run it the same way

```
uv run lessons/01-model-interface/model_interface.py
```

One run: the **thinking** block

```
"content": [  
  {  
    "signature": "CAQS0AUKEAgSGAI4AUIIdGhpbmtpbmcSDJo1A0U3Q8nzeKwLQRoMqA3M2GGFMP3dZLX+IjCE...",  
    "thinking": "I don't actually have access to this user's file system, so I can't see what's in their directory—I should explain that and suggest they paste the listing or output here instead.\n\n",  
    "type": "thinking"  
  },  
]
```

The **signature** is shortened here too.

One difference

It looks like the first response (its `signature` shortened too), with one difference: the `thinking` block has text.

That's the summary `display: "summarized"` asked for.

The rest shows when something goes wrong

What this version adds only shows when something goes wrong:

- a dropped connection
- a rate limit
- a model that's down

What to take away

The rule: the model interface sends tokens to the model as a request and gets tokens back as a response.

What the model interface never does

- When to call is **control flow**
- Gathering what goes in, from a person or the world, is **input**
- How it's presented in the request is **context**
- What happens to the response is **output**

It only gets the request there

The model interface only gets the request there and the response back.

What's missing

Both ends of the path are stubs.

```
"What's in this directory?" → request → model interface → response → print(...)
input (a stub)                                     output (a stub)
```

The two stubs

The input is a hardcoded string no one typed.

The output dumps the whole response without handling any of it.

The model said what to do

The model said what to do, and nothing could do it.

Making both ends real is input and output.

Next: Input and output

Lesson 2